Experiment 1.1

Aim :

To write a C program to implement **Hashing** using different collision resolution techniques such as **Separate Chaining, Linear Probing, and Quadratic Probing** with operations like **insertion, searching, deletion, and display** using a **menu-driven approach**.

Description :

Hashing is a technique used to **store and retrieve data efficiently** using a **hash table**. A hash function maps a given key to a specific index in the hash table.

A commonly used hash function is:

h(key) = key % table_size

Sometimes two different keys may produce the **same index**. This situation is called a **collision**. To handle collisions, different **collision resolution techniques** are used.

1. Separate Chaining

Separate chaining is an **open hashing technique** where each index of the hash table contains a **linked list**. If multiple keys map to the same index, they are stored in the linked list at that position.

2. Linear Probing

Linear probing is an **open addressing technique** where the next available position is searched sequentially.

index = (h(key) + i) % table_size

It is simple to implement but may lead to **primary clustering**.

3. Quadratic Probing

Quadratic probing resolves collisions using a quadratic function. **index =**

(h(key) + i^2) % table_size

It reduces clustering compared to linear probing.

Applications of Hashing

- Symbol tables in compilers
- Databases
- Caches
- Password storage systems

Procedure / Algorithm with Complexities :

Algorithm

1. Start the program.
2. Define a hash table with a fixed size.
3. Initialize all positions in the hash table as empty.
4. Display the following menu options:
   - Insert element
   - Search element
   - Delete element
   - Display hash table
   - Exit
5. Read the user's choice.
6. Calculate the hash index using **index = key % table_size**
7. If collision occurs:
   - Use **Separate Chaining** by inserting the key into a linked list.
   - Use **Linear Probing** by checking (index + i) % table_size.
   - Use **Quadratic Probing** by checking (index + i^2) % table_size.
8. Perform insert, search, delete or display operations.
9. Repeat until the user chooses exit.
10. Stop the program.

Time Complexity

Insertion : **O(1)** average
Search : **O(1)** average
Deletion : **O(1)** average Worst
Case : **O(n)**

Space Complexity: O(n)

Source Code :

```
#include <stdio.h>
#include <stdlib.h> #include
<limits.h>

#define SIZE 10
#define EMPTY INT_MIN
#define DELETED (INT_MIN + 1)

/* ================= CHAINING ================= */

struct Node {
   int data;     struct
Node *next;
};

struct Node *chainTable[SIZE];

int hashFunction(int key){
   return key % SIZE;
}

void initChain(){
   int i;
   for(i=0;i<SIZE;i++)
      chainTable[i] = NULL;
}

void chainInsert(int key){     int
index = hashFunction(key);

   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));     newNode->data =
key;
   newNode->next = chainTable[index];     chainTable[index]
= newNode;
```

```
   printf("Inserted %d in chain index %d\n",key,index);
}

void chainSearch(int key){
   int index = hashFunction(key);

   struct Node *temp = chainTable[index];

   while(temp){        if(temp-
>data == key){
          printf("Key found in chain index %d\n",index);
return;
       }
       temp = temp->next;
   }

   printf("Key not found\n");
}

void chainDelete(int key){
   int index = hashFunction(key);

   struct Node *temp = chainTable[index];
   struct Node *prev = NULL;

   while(temp){

       if(temp->data == key){

          if(prev == NULL)
chainTable[index] = temp->next;
          else
             prev->next = temp->next;

          free(temp);

          printf("Key deleted\n");
return;
       }

       prev = temp;
```

```
        temp = temp->next;
    }

    printf("Key not found\n");
}

void displayChain(){

    int i;
    struct Node *temp;

    for(i=0;i<SIZE;i++){

        printf("%d : ",i);

        temp = chainTable[i];

        while(temp){

            printf("%d -> ",temp->data);
temp = temp->next;
        }

        printf("NULL\n");
    }
}

/* ================= LINEAR PROBING ================= */

int linearTable[SIZE];

void initLinear(){

    int i;

    for(i=0;i<SIZE;i++)
        linearTable[i] = EMPTY;
}

void linearInsert(int key){
    int index = hashFunction(key), i;
```

```
    for(i=0;i<SIZE;i++){

        int probe = (index + i) % SIZE;

        if(linearTable[probe] == EMPTY || linearTable[probe] == DELETED){

            linearTable[probe] = key;

            printf("Inserted at %d\n",probe);

            return;
        }
    }

    printf("Table full\n");
}

void linearSearch(int key){

    int index = hashFunction(key), i;

    for(i=0;i<SIZE;i++){

        int probe = (index + i) % SIZE;

        if(linearTable[probe] == EMPTY)
            break;

        if(linearTable[probe] == key){

            printf("Found at index %d\n",probe);

            return;
        }
    }

    printf("Not found\n");
}

void linearDelete(int key){
```

```
    int index = hashFunction(key), i;

    for(i=0;i<SIZE;i++){

        int probe = (index + i) % SIZE;

        if(linearTable[probe] == EMPTY)
            break;

        if(linearTable[probe] == key){

            linearTable[probe] = DELETED;

            printf("Deleted from %d\n",probe);

            return;
        }
    }

    printf("Key not found\n");
}

void displayLinear(){

    int i;

    for(i=0;i<SIZE;i++){

        if(linearTable[i] == EMPTY)
            printf("[%d] EMPTY\n",i);

        else if(linearTable[i] == DELETED)
printf("[%d] DELETED\n",i);

        else
            printf("[%d] %d\n",i,linearTable[i]);
    }
}

/* ================= QUADRATIC PROBING ================= */
```

```
int quadTable[SIZE];

void initQuad(){

   int i;

   for(i=0;i<SIZE;i++)
      quadTable[i] = EMPTY;
}

void quadInsert(int key){

   int index = hashFunction(key), i;

   for(i=0;i<SIZE;i++){

      int probe = (index + i*i) % SIZE;

      if(quadTable[probe] == EMPTY || quadTable[probe] == DELETED){

         quadTable[probe] = key;

         printf("Inserted at %d\n",probe);

         return;
      }
   }

   printf("Table full\n");
}

void quadSearch(int key){

   int index = hashFunction(key), i;

   for(i=0;i<SIZE;i++){

      int probe = (index + i*i) % SIZE;
      if(quadTable[probe] == EMPTY)
         break;
```

```
        if(quadTable[probe] == key){

            printf("Found at index %d\n",probe);

            return;
        }
    }

    printf("Not found\n");
}

void quadDelete(int key){

    int index = hashFunction(key), i;

    for(i=0;i<SIZE;i++){

        int probe = (index + i*i) % SIZE;

        if(quadTable[probe] == EMPTY)
            break;

        if(quadTable[probe] == key){

            quadTable[probe] = DELETED;

            printf("Deleted from %d\n",probe);

            return;
        }
    }

    printf("Key not found\n");
}

void displayQuad(){

    int i;
    for(i=0;i<SIZE;i++){
```

```
        if(quadTable[i] == EMPTY)
printf("[%d] EMPTY\n",i);

        else if(quadTable[i] == DELETED)
            printf("[%d] DELETED\n",i);

        else
            printf("[%d] %d\n",i,quadTable[i]);
    }
}

/* ================= MAIN MENU ================= */

int main(){

    int choice,key,tech;

    initChain();
initLinear();     initQuad();

    while(1){

        printf("\nChoose Technique\n");
printf("1. Separate Chaining\n");
printf("2. Linear Probing\n");        printf("3.
Quadratic Probing\n");
        printf("4. Exit\n");

        scanf("%d",&tech);

        if(tech==4)
            break;

        printf("\n1.Insert\n2.Search\n3.Delete\n4.Display\n");

        scanf("%d",&choice);

        if(choice!=4){
            printf("Enter key: ");
scanf("%d",&key);
        }
```

```
    switch(tech){

    case 1:

      if(choice==1)
chainInsert(key);

      else if(choice==2)
        chainSearch(key);

      else if(choice==3)
        chainDelete(key);

      else if(choice==4)
        displayChain;

      break;

    case 2:

      if(choice==1)
        linearInsert(key);

      else if(choice==2)
        linearSearch(key);

      else if(choice==3)
        linearDelete(key);

      else if(choice==4)
displayLinear();

      break;

    case 3:

      if(choice==1)
quadInsert(key);        else
if(choice==2)
        quadSearch(key);
```

```
        else if(choice==3)
            quadDelete(key);

        else if(choice==4)
            displayQuad();

        break;

    default:

        printf("Invalid option\n");
    }
  }

  return 0;
}
```

INPUT & OUTPUT :

```
Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
1

1.Insert
2.Search
3.Delete
4.Display

Enter key: 25
Inserted 25 in chain index 5

Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
1

1.Insert
2.Search
3.Delete
4.Display
4

0 : NULL
1 : NULL
2 : NULL
3 : NULL
4 : NULL
5 : 25 -> NULL
6 : NULL
7 : NULL
8 : NULL
9 : NULL
```

```
Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
2

1.Insert
2.Search
3.Delete
4.Display

Enter key: 45
Inserted at 5

Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
2

1.Insert
2.Search
3.Delete
4.Display
4

[0] EMPTY
[1] EMPTY
[2] EMPTY
[3] EMPTY
[4] EMPTY
[5] 45
[6] EMPTY
[7] EMPTY
[8] EMPTY
[9] EMPTY
```

```
Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
3

1.Insert
2.Search
3.Delete
4.Display

Enter key: 35
Inserted at 5

Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
3

1.Insert
2.Search
3.Delete
4.Display
4

[0] EMPTY
[1] EMPTY
[2] EMPTY
[3] EMPTY
[4] EMPTY
[5] 35
[6] EMPTY
[7] EMPTY
[8] EMPTY
[9] EMPTY


Choose Technique
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
4
```

Result : Thus, the program to implement hashing using Separate Chaining, Linear Probing, and Quadratic Probing with insertion, searching, deletion, and display operations was successfully implemented and executed.

Experiment 2.1

Aim :

To write a C program to implement **AVL Tree operations** such as insertion, deletion, searching, and tree traversals (**inorder, preorder, and postorder**) using a **menu-driven approach**.

Description :

An **AVL Tree** is a self-balancing **Binary Search Tree (BST)** where the difference between the heights of the left and right subtrees of any node is at most **1**.

This difference is known as the **Balance Factor**.

Balance Factor = Height(Left Subtree) − Height(Right Subtree) The

balance factor of each node must be **-1, 0, or +1**.

If the tree becomes unbalanced after **insertion or deletion**, rotations are performed to restore the balance of the tree.

Types of Rotations

1. **LL Rotation (Left-Left Case)** – Right rotation
2. **RR Rotation (Right-Right Case)** – Left rotation
3. **LR Rotation (Left-Right Case)** – Left rotation followed by right rotation
4. **RL Rotation (Right-Left Case)** – Right rotation followed by left rotation

Advantages

- Maintains balanced height of the tree
- Faster searching compared to normal BST
- Time complexity remains **O(log n)**

Applications

- Database indexing
- Memory management systems
- Efficient searching in large datasets

Procedure / Algorithm with Complexities :

Algorithm

1. Start the program.
2. Create a node structure containing **key, left pointer, right pointer, and height**.
3. Display the menu with the following options:

- Insert node
- Delete node
- Search node
- Inorder traversal
- Preorder traversal
- Postorder traversal
- Exit

4. For **Insertion**:
   - Insert the node according to **Binary Search Tree rules**. ○ Update the height of the node. ○ Calculate the balance factor.
   - Perform necessary rotations (**LL, RR, LR, RL**) if the tree becomes unbalanced.
5. For **Deletion**:
   - Delete the node using **BST deletion rules**. ○ Update height and balance factor.
   - Perform required rotations to maintain AVL balance.
6. For **Searching**:
   - Compare the key with the root. ○ Move to the left subtree if the key is smaller.
   - Move to the right subtree if the key is larger.
7. For **Traversal**:

- Perform **Inorder traversal** to display elements in sorted order.
- Perform **Preorder traversal** to display root before subtrees.
- Perform **Postorder traversal** to display root after subtrees.

8. Repeat the process until the user selects **Exit**.
9. Stop the program.

Time Complexity

Insertion : **O(log n)**
Deletion : **O(log n)**
Search : **O(log n)**
Traversal : **O(n)**

Space Complexity : O(n)

Source Code :

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int key;     struct node
*left,*right;
   int height;
};

int max(int a,int b){
return (a>b)?a:b;
}

int height(struct node *n){
if(n==NULL)
       return 0;
   return n->height;
}

struct node* newnode(int key){
```

```
   struct node* node=(struct node*)malloc(sizeof(struct node));
node->key=key;     node->left=node->right=NULL;
   node->height=1;
   return node;
}

struct node* minValueNode(struct node* node){
   struct node* current=node;
while(current->left!=NULL)
current=current->left;     return
current;
}

struct node* rightRotate(struct node* y){
   struct node* x=y->left;
struct node* T2=x->right;

   x->right=y;
   y->left=T2;

   y->height=max(height(y->left),height(y->right))+1;
   x->height=max(height(x->left),height(x->right))+1;

   return x;
}

struct node* leftRotate(struct node* x){
struct node* y=x->right;     struct node*
T2=y->left;

   y->left=x;
   x->right=T2;

   x->height=max(height(x->left),height(x->right))+1;
   y->height=max(height(y->left),height(y->right))+1;

   return y;
}

int getBalance(struct node* node){     if(node==NULL)
       return 0;
```

```
    return height(node->left)-height(node->right);
}

struct node* insertNode(struct node* node,int key){

    if(node==NULL){
        printf("Node %d inserted successfully\n",key);
return newnode(key);
    }

    if(key<node->key)
        node->left=insertNode(node->left,key);

    else if(key>node->key)
        node->right=insertNode(node->right,key);

    else{
        printf("Duplicate key not allowed\n");
        return node;
    }

    node->height=1+max(height(node->left),height(node->right));

    int balance=getBalance(node);

    if(balance>1 && key<node->left->key)
return rightRotate(node);

    if(balance<-1 && key>node->right->key)
        return leftRotate(node);

    if(balance>1 && key>node->left->key){
node->left=leftRotate(node->left);        return
rightRotate(node);
    }

    if(balance<-1 && key<node->right->key){        node->right=rightRotate(node-
>right);
        return leftRotate(node);
    }
```

```
    return node;
}

struct node* deleteNode(struct node* root,int key){

    if(root==NULL){
printf("Key not found\n");
return root;
    }

    if(key<root->key)
        root->left=deleteNode(root->left,key);

    else if(key>root->key)
        root->right=deleteNode(root->right,key);

    else{

        printf("Node %d deleted successfully\n",key);

        if(root->left==NULL||root->right==NULL){

            struct node* temp=root->left?root->left:root->right;

            if(temp==NULL){
temp=root;              root=NULL;
            }
            else
                *root=*temp;

            free(temp);
        }

        else{

            struct node* temp=minValueNode(root->right);

            root->key=temp->key;

            root->right=deleteNode(root->right,temp->key);
        }
```

```
    }

    if(root==NULL)
return root;

    root->height=1+max(height(root->left),height(root->right));     int balance=getBalance(root);

    if(balance>1 && getBalance(root->left)>=0)
        return rightRotate(root);

    if(balance>1 && getBalance(root->left)<0){        root->left=leftRotate(root-
>left);
        return rightRotate(root);
    }

    if(balance<-1 && getBalance(root->right)<=0)
        return leftRotate(root);

    if(balance<-1   &&  getBalance(root->right)>0){
root->right=rightRotate(root->right);          return
leftRotate(root);
    }

    return root;
}

struct node* search(struct node* root,int key){

    if(root==NULL||root->key==key)
        return root;

    if(key<root->key)        return
search(root->left,key);

    return search(root->right,key);
}

void inorder(struct node* root){

    if(root!=NULL){
inorder(root->left);        printf("%d
```

```
",root->key);        inorder(root-
>right);
   }
}

void preorder(struct node* root){    if(root!=NULL){        printf("%d ",root->key);        preorder(root->left);
preorder(root->right);
   }
}

void postorder(struct node* root){

   if(root!=NULL){        postorder(root-
>left);        postorder(root->right);
      printf("%d ",root->key);
   }
}

int main(){

   struct node* root=NULL;

   int choice,key;

   while(1){

      printf("\n\n--- AVL TREE MENU ---");
printf("\n1.Insert");        printf("\n2.Delete");
printf("\n3.Search");        printf("\n4.Inorder
Traversal");        printf("\n5.Preorder
Traversal");        printf("\n6.Postorder
Traversal");        printf("\n7.Exit");

      printf("\nEnter choice: ");
      scanf("%d",&choice);

      switch(choice){

         case 1:
         printf("Enter key to insert: ");
scanf("%d",&key);           root=insertNode(root,key);
```

```
        break;

        case 2:
        printf("Enter key to delete: ");
scanf("%d",&key);          root=deleteNode(root,key);
        break;

        case 3:
        printf("Enter key to search: ");
        scanf("%d",&key);

        if(search(root,key))
printf("Key %d FOUND\n",key);
        else
           printf("Key %d NOT FOUND\n",key);
        break;

        case 4:
        printf("Inorder Traversal: ");
inorder(root);          break;

        case 5:
        printf("Preorder Traversal: ");
preorder(root);          break;

        case 6:
        printf("Postorder Traversal: ");
postorder(root);          break;

        case 7:
        printf("Exiting program\n");
        exit(0);

        default:
        printf("Invalid choice\n");
      }
   }
}
```

Input & Output

```
--- AVL TREE MENU ---
1.Insert
2.Delete
3.Search
4.Inorder Traversal
5.Preorder Traversal
6.Postorder Traversal
7.Exit

Enter choice: 1
Enter key to insert: 30
Node 30 inserted successfully

Enter choice: 1
Enter key to insert: 20
Node 20 inserted successfully

Enter choice: 1
Enter key to insert: 40
Node 40 inserted successfully

Enter choice: 4
Inorder Traversal: 20 30 40

Enter choice: 3
Enter key to search: 20
Key 20 FOUND

Enter choice: 2
Enter key to delete: 20
Node 20 deleted successfully

Enter choice: 4
Inorder Traversal: 30 40

Enter choice: 7
Exiting program
```

Result :

Thus, the program to implement AVL Tree operations including insertion, deletion, searching, and tree traversals using a menu-driven approach was successfully implemented and executed.

Experiment 2.2 Aim :

To write a C program to implement **Max Heap and Min Heap operations** such as insertion, deletion, and display using a **menu-driven approach**.

Description :

A **Heap** is a special type of **complete binary tree** that satisfies the heap property. Heaps are commonly implemented using **arrays**.

There are two types of heaps.

1. Max Heap

In a **Max Heap**, the value of the parent node is **greater than or equal to** the values of its children. Therefore, the **largest element is always at the root**.

Example:

```
    50
   / \
  30   40
 / \
10  20
```

2. Min Heap

In a **Min Heap**, the value of the parent node is **less than or equal to** the values of its children. Therefore, the **smallest element is always at the root**.

Example:

```
    10
   / \
  20   30
 / \
40  50
```

Properties of Heap

- It is a **complete binary tree**.
- Usually implemented using **arrays**.

Parent and child relationship:

Left Child = **2*i + 1**
Right Child = **2*i + 2**
Parent = **(i - 1) / 2**

Applications

- Priority queues
- Heap sort
- Graph algorithms like **Dijkstra and Prim**

Procedure / Algorithm with Complexities :

Algorithm

1. Start the program.
2. Define arrays to represent **Max Heap and Min Heap**.
3. Display the menu options:

- Insert element
- Delete element
- Display heap
- Exit

4. For **Insertion**:
   - Insert the element at the end of the heap. ○ Compare the inserted element with its parent.
   - Swap if the heap property is violated.
5. For **Deletion**:
   - Remove the root element. ○ Replace the root with the last element.
   - Heapify the tree to maintain heap property.
6. For **Display**:
   - Print all elements in the heap array.
7. Repeat until the user selects **Exit**.
8. Stop the program.

Time Complexity

Insertion : **O(log n)**
Deletion : **O(log n)** Display :
O(n)

Space Complexity: O(n)

Source Code :

```
#include <stdio.h>

#define SIZE 100

int maxHeap[SIZE], minHeap[SIZE];
int maxSize = 0, minSize = 0;

/* -------- MAX HEAP -------- */

void insertMax(int value)
{
   int i = maxSize++;
   maxHeap[i] = value;

   while(i != 0 && maxHeap[(i-1)/2] < maxHeap[i])
   {
      int temp = maxHeap[i];
maxHeap[i] = maxHeap[(i-1)/2];
maxHeap[(i-1)/2] = temp;

      i = (i-1)/2;
   }
}

void deleteMax()
{
   if(maxSize <= 0)
   {
      printf("Max Heap is empty\n");
      return;
   }

   printf("Deleted element: %d\n", maxHeap[0]);

   maxHeap[0] = maxHeap[--maxSize];
```

```
   int i = 0;

   while(1)
   {
      int left = 2*i + 1;
int right = 2*i + 2;
int largest = i;

      if(left < maxSize && maxHeap[left] > maxHeap[largest])
largest = left;

      if(right < maxSize && maxHeap[right] > maxHeap[largest])
largest = right;

      if(largest != i)
      {
         int temp = maxHeap[i];
maxHeap[i] = maxHeap[largest];
         maxHeap[largest] = temp;

         i = largest;
      }
      else
break;
   }
}

void displayMax()
{    int
i;

   printf("Max Heap: ");

   for(i=0;i<maxSize;i++)
      printf("%d ", maxHeap[i]);

   printf("\n");
}

/* -------- MIN HEAP -------- */
```

```
void insertMin(int value)
{
   int i = minSize++;
   minHeap[i] = value;

   while(i != 0 && minHeap[(i-1)/2] > minHeap[i])
   {
      int temp = minHeap[i];
minHeap[i] = minHeap[(i-1)/2];
      minHeap[(i-1)/2] = temp;

      i = (i-1)/2;
   }
}

void deleteMin()
{
   if(minSize <= 0)
   {
      printf("Min Heap is empty\n");
      return;
   }

   printf("Deleted element: %d\n", minHeap[0]);

   minHeap[0] = minHeap[--minSize];

   int i = 0;

   while(1)
   {
      int left = 2*i + 1;
int right = 2*i + 2;
      int smallest = i;

     if(left < minSize && minHeap[left] < minHeap[smallest])
smallest = left;

      if(right < minSize && minHeap[right] < minHeap[smallest])
smallest = right;
```

```
        if(smallest != i)
        {
            int temp = minHeap[i];
minHeap[i] = minHeap[smallest];
            minHeap[smallest] = temp;

            i = smallest;
        }
        else
            break;
    }
}

void displayMin()
{    int
i;

    printf("Min Heap: ");

    for(i=0;i<minSize;i++)
        printf("%d ", minHeap[i]);

    printf("\n");
}

/* -------- MAIN -------- */

int main()
{
    int choice, value;

    while(1)
    {
        printf("\n--- HEAP MENU ---\n");
printf("1 Insert in Max Heap\n");        printf("2
Delete from Max Heap\n"); printf("3 Display
Max Heap\n");
        printf("4 Insert in Min Heap\n");
printf("5 Delete from Min Heap\n");
```

```
printf("6 Display Min Heap\n");        printf("7
Exit\n");

      printf("Enter choice: ");
      scanf("%d",&choice);

      switch(choice)
      {
         case 1:
printf("Enter value: ");
scanf("%d",&value);
insertMax(value);
            break;

         case 2:
deleteMax();
break;

         case 3:
displayMax();
            break;

         case 4:
            printf("Enter value: ");
scanf("%d",&value);              insertMin(value);
            break;

         case 5:
deleteMin();
            break;

         case 6:
displayMin();
break;

         case 7:
return 0;
default:
            printf("Invalid choice\n");
      }
```

```
    }
}
```

Input & Output :

```
--- HEAP MENU ---
1 Insert in Max Heap
2 Delete from Max Heap
3 Display Max Heap
4 Insert in Min Heap
5 Delete from Min Heap
6 Display Min Heap
7 Exit

Enter choice: 1
Enter value: 50

Enter choice: 1
Enter value: 30

Enter choice: 1
Enter value: 40

Enter choice: 3
Max Heap: 50 30 40

Enter choice: 2
Deleted element: 50

Enter choice: 3
Max Heap: 40 30


Enter choice: 4
Enter value: 20

Enter choice: 4
Enter value: 10

Enter choice: 4
Enter value: 30

Enter choice: 6
Min Heap: 10 20 30

Enter choice: 5
Deleted element: 10

Enter choice: 6
Min Heap: 20 30

Enter choice: 7
```

Result

Thus the program to implement **Max Heap and Min Heap operations such as insertion, deletion, and display** was **successfully executed**.

Experiment 3.1

Aim :

To write a C program to **sort a list of elements using the Merge Sort technique** based on the **Divide and Conquer approach**.

Description :

Merge Sort is a **sorting algorithm based on the Divide and Conquer technique**.
The algorithm works by repeatedly dividing the array into two halves until each subarray contains a single element. These subarrays are then merged in a sorted manner to produce the final sorted array.

Steps of Merge Sort

1. **Divide**: Split the array into two halves.
2. **Conquer**: Recursively sort both halves.
3. **Combine**: Merge the two sorted halves into a single sorted array.

Properties

- Stable sorting algorithm
- Works efficiently for large datasets
- Uses recursion
- Requires extra memory for merging

Procedure / Algorithm with Complexities :

Algorithm

1. Start the program.
2. Read the number of elements in the array.
3. Input the elements of the array.
4. Divide the array into two halves.
5. Recursively apply **Merge Sort** to both halves.
6. Merge the two sorted halves into a single sorted array.

7. Repeat the process until the entire array becomes sorted.
8. Display the sorted array.
9. Stop the program.

Time Complexity

Best Case : **O(n log n)**
Average Case : **O(n log n)** Worst
Case : **O(n log n)**

Space Complexity

O(n) (extra space required for merging)

Source Code :

```
#include <stdio.h>

void merge(int arr[], int l, int m, int r)
{    int i, j,
k;

   int n1 = m - l + 1;
   int n2 = r - m;

   int L[n1], R[n2];

   for(i = 0; i < n1; i++)
      L[i] = arr[l + i];

   for(j = 0; j < n2; j++)
R[j] = arr[m + 1 + j];

   i = 0;
j = 0;
k = l;

   while(i < n1 && j < n2)
   {
```

```
        if(L[i] <= R[j])
        {
            arr[k] = L[i];
i++;
        }
        else
        {
            arr[k] = R[j];
j++;
        }
        k++;
    }

    while(i < n1)
    {
        arr[k] = L[i];
i++;
        k++;
    }

    while(j < n2)
    {
        arr[k] =
R[j];        j++;
k++;
    }
}

void mergeSort(int arr[], int l, int r)
{    if(l <
r)
    {
        int m = (l + r) / 2;

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}
```

```
int main()
{
   int arr[100], n, i;

   printf("Enter number of elements: ");
scanf("%d", &n);

   printf("Enter elements:\n");

   for(i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   mergeSort(arr, 0, n - 1);

   printf("Sorted array:\n");

   for(i = 0; i < n; i++)
printf("%d ", arr[i]);

   return 0;
}
```

Input & Output :

```
Enter number of elements: 5
Enter elements:
34 12 25 9 56

Sorted array:
9 12 25 34 56
```

Result :

Thus, the program to **sort the given list of elements using the Merge Sort technique based on the Divide and Conquer approach** was successfully implemented and executed.

Experiment 3.2 Aim :

To write a C program to implement **Single Source Shortest Path using Dijkstra's Algorithm** based on the **Greedy approach**.

Description :

The **Single Source Shortest Path problem** determines the shortest distance from a **single source vertex to all other vertices** in a weighted graph.

One of the most commonly used algorithms to solve this problem is **Dijkstra's Algorithm**, which follows a **greedy approach**.

In this algorithm, the vertex with the **minimum tentative distance from the source** is selected at each step. The distances to its neighboring vertices are then updated if a **shorter path** is found.

The algorithm continues until the shortest distance from the source vertex to **all other vertices** is determined.

Working Principle

1. Assign distance value **0** to the source vertex.
2. Assign **infinite distance** to all other vertices.
3. Select the vertex with the **minimum distance** that has not been visited.
4. Update the distance of its adjacent vertices.
5. Repeat the process until **all vertices are processed**.

Applications

• Network routing algorithms
• GPS navigation systems
• Communication networks
• Transportation systems

Procedure / Algorithm with Complexities :

Algorithm

1. Start the program.
2. Read the number of vertices in the graph.
3. Input the **cost adjacency matrix** of the graph.
4. Replace the value **0 with infinity** where there is no direct edge.
5. Read the **source vertex**.
6. Initialize all vertices as **unvisited**.
7. Set the distance of the **source vertex to 0** and all others to infinity.
8. Select the **unvisited vertex with the minimum distance**.
9. Mark the selected vertex as **visited**.
10. Update the distances of adjacent vertices if a **shorter path** is found.
11. Repeat steps **8–10** until all vertices are visited.
12. Display the shortest distance from the **source to all other vertices**.
13. Stop the program.

Time Complexity

$O(V^2)$ when using an adjacency matrix.

Space Complexity: O(V)

Source Code :

```
#include<stdio.h>

#define infinity 999

void dijkstra(int n, int v, int cost[10][10], int dist[])
{
   int i, u, count, w, flag[10], min;
```

```
   for(i = 0; i < n; i++)
   {
      flag[i] = 0;
dist[i] = cost[v][i];     }

   dist[v] = 0;
flag[v] = 1;     count
= 1;

   while(count < n)
   {
      min = infinity;
u = -1;

      for(w = 0; w < n; w++)
      {
         if(!flag[w] && dist[w] < min)
         {
            min = dist[w];
            u = w;
         }
      }

      if(u == -1)
         break;

      flag[u] = 1;
count++;

      for(w = 0; w < n; w++)
      {
         if(!flag[w] && (dist[u] + cost[u][w] < dist[w]))
         {
            dist[w] = dist[u] + cost[u][w];
         }
      }
   }
}

int main()
```

```
{    int
n,v,i,j;
   int cost[10][10], dist[10];

   printf("Enter the number of nodes:\n");     scanf("%d",&n);

   printf("Enter the cost matrix (use 0 for no direct path):\n");

   for(i =0; i<n ; i++)
   {
      for(j = 0; j < n; j++)
      {
         scanf("%d",&cost[i][j]);

         if(cost[i][j] == 0)
            cost[i][j] = infinity;
      }
   }

   printf("Enter the source node (0 to %d):\n",n-1);
scanf("%d",&v);

   if(v < 0 || v >= n)
   {
      printf("Invalid source node.\n");
      return 1;
   }

   dijkstra(n, v, cost, dist);

   printf("Shortest paths from node %d:\n",v);

   for(i = 0; i < n; i++)
   {        if(i
!= v)
      {
         printf("%d -> %d, cost = %d\n", v, i, dist[i]);
      }
   }
    return 0;
}
```

Input & Output :

```
Enter the number of nodes:
4

Enter the cost matrix (use 0 for no direct path):

0 2 4 0
2 0 1 7
4 1 0 3
0 7 3 0

Enter the source node (0 to 3):
0

Shortest paths from node 0:
0 -> 1, cost = 2
0 -> 2, cost = 3
0 -> 3, cost = 6
```

Result :

Thus, the program to implement **Single Source Shortest Path using Dijkstra's Algorithm based on the Greedy approach** was successfully implemented and executed.

Experiment 4.1

Aim :
To write a C program to solve the **0/1 Knapsack problem using the Dynamic Programming approach** and determine the **maximum profit without exceeding the knapsack capacity**.

Description :
The **0/1 Knapsack problem** is an optimization problem where a set of items with **weights and values** is given. The objective is to select items such that the **total profit is maximized while the total weight does not exceed the knapsack capacity**.
Each item can either be **included or excluded**, meaning items cannot be divided. Dynamic Programming solves this problem by storing intermediate results in a table and building the final solution step by step.

Algorithm
1. Start the program.
2. Read the number of items **n**.
3. Input the **weights and values** of the items.
4. Read the knapsack capacity **W**.
5. Create a DP table **K[n+1][W+1]**.
6. Initialize the first row and first column with **0**.
7. For each item **i** and capacity **w**:
   - If the weight of the item is less than or equal to **w**, compute the maximum of including or excluding the item.
   - Otherwise copy the value from the previous row.
8. Repeat the process for all items and capacities.
9. The value stored in **K[n][W]** gives the maximum profit.
10. Display the result.
11. Stop the program.

Time Complexity : O(n × W)
Space Complexity: O(n × W)

Source Code :
```
#include <stdio.h>

int max(int a,int b){
   return (a>b)?a:b;
}

void knapsack(int W,int wt[],int val[],int n){
   int i,w;
   int K[n+1][W+1];
   for(i=0;i<=n;i++){
      for(w=0;w<=W;w++){
```

```
        if(i==0||w==0)
            K[i][w]=0;
        else if(wt[i-1]<=w)
            K[i][w]=max(val[i-1]+K[i-1][w-wt[i-1]],K[i-1][w]);
        else
            K[i][w]=K[i-1][w];
    }
  }
  printf("Maximum profit: %d\n",K[n][W]);
}
int main(){
  int n,W,i;
  int wt[10],val[10];

  printf("Enter number of items: ");
  scanf("%d",&n);

  printf("Enter weights and values:\n");
  for(i=0;i<n;i++)
    scanf("%d %d",&wt[i],&val[i]);

  printf("Enter capacity of knapsack: ");
  scanf("%d",&W);

  knapsack(W,wt,val,n);

  return 0;
}
```

Input & Output :

```
Enter number of items: 3
Enter weights and values:
10 60
20 100
30 120
Enter capacity of knapsack: 50
Maximum profit: 220
```

Result :

Thus, the program to solve the **0/1 Knapsack problem using Dynamic Programming** was successfully implemented and executed.

Experiment 4.2

Aim :

To write a C program to find the shortest path between all pairs of vertices in a graph using the Floyd–Warshall Algorithm based on the Dynamic Programming approach.

Description :

The All Pairs Shortest Path problem finds the minimum distance between every pair of vertices in a weighted graph.

The Floyd–Warshall Algorithm uses Dynamic Programming to update the shortest path by considering each vertex as an intermediate node.

If a shorter path between two vertices exists through another vertex, the distance is updated. This process continues until the shortest distances between all vertex pairs are obtained.

Algorithm

1. Start the program.
2. Read the number of vertices **n**.
3. Input the **adjacency matrix** representing the graph.
4. Initialize the distance matrix with the given values.
5. Consider each vertex **k** as an intermediate vertex.
6. For each pair of vertices **i** and **j**, update the distance using:

dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

7. Repeat the process for all vertices.
8. Display the final shortest path matrix.
9. Stop the program.

Time Complexity $O(n^3)$

Space Complexity $O(n^2)$

Source Code :

```
#include <stdio.h>

#define INF 99999

int main() {
   int n,i,j,k;
   printf("Enter number of vertices: ");
   scanf("%d",&n);
   int dist[n][n];
   printf("Enter adjacency matrix (use %d for no path):\n",INF);

   for(i=0;i<n;i++)
      for(j=0;j<n;j++)
         scanf("%d",&dist[i][j]);
```

```
    for(k=0;k<n;k++){
        for(i=0;i<n;i++){
            for(j=0;j<n;j++){
                if(dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
    printf("\nAll-Pairs Shortest Path Matrix:\n");
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            if(dist[i][j]==INF)
                printf("INF ");
            else
                printf("%d ",dist[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Input & Output :

```
Enter number of vertices: 4

Enter adjacency matrix:
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0

All-Pairs Shortest Path Matrix:

0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
```

Result :

Thus, the program to find the **shortest paths between all pairs of vertices using the Floyd–Warshall Algorithm** was successfully implemented and executed.

Experiment 5.1

Aim :
To write a C program to solve the **N-Queens problem using the Backtracking technique** and place **N queens on an N × N chessboard** such that no two queens attack each other.

Description :
The N-Queens problem is a classic problem where N queens must be placed on a chessboard so that no queen attacks another queen.
This means no two queens should be in the same row, column, or diagonal.
The Backtracking algorithm places queens one by one and checks if the position is safe. If a conflict occurs, the algorithm removes the queen and tries another position until a valid solution is found.

Algorithm
1. Start the program.
2. Read the value of **N** (number of queens).
3. Begin placing queens from the first row.
4. For each column in the row, check whether the position is safe.
5. If safe, place the queen and move to the next row.
6. If no safe position exists, backtrack to the previous row.
7. Continue this process until all queens are placed.
8. Display the valid solution.
9. Stop the program.

Time Complexity - Worst Case : **O(N!)**
Space Complexity: O(N)

Source Code :
```
#include<stdio.h>
#include<math.h>

int board[20],count;

void queen(int row,int n);
int place(int row,int column);
void print(int n);

int main()
{
   int n;

   printf("N Queens Problem Using Backtracking\n");
   printf("Enter number of Queens: ");
   scanf("%d",&n);
```

```
    queen(1,n);

    printf("\nTotal Solutions with %d Queens: %d",n,count);

    return 0;
}

void print(int n)
{
    int i;

    printf("\nSolution %d: ",++count);

    for(i=1;i<=n;i++)
        printf("%d ",board[i]);
}

int place(int row,int column)
{
    int i;

    for(i=1;i<=row-1;i++)
    {
        if(board[i]==column || abs(board[i]-column)==abs(i-row))
            return 0;
    }

    return 1;
}

void queen(int row,int n)
{
    int column;

    for(column=1;column<=n;column++)
    {
        if(place(row,column))
        {
            board[row]=column;

            if(row==n)
                print(n);
```

```
        else
           queen(row+1,n);
     }
   }
}
```

Input & Output :

```
Enter number of Queens: 4

Solution 1: 2 4 1 3
Solution 2: 3 1 4 2

Total Solutions with 4 Queens: 2
```

Result :

Thus, the program to solve the **N-Queens problem using the Backtracking algorithm** was successfully implemented and executed.

Experiment 5.2

Aim :
To write a C program to solve the **Travelling Salesperson Problem (TSP)** using the **Branch and Bound approach** and determine the **minimum cost tour**.

Description :
The Travelling Salesperson Problem (TSP) is a problem where a salesperson must visit all cities exactly once and return to the starting city while minimizing the total travel cost.
The Branch and Bound technique explores possible paths and eliminates paths whose cost exceeds the current minimum cost, thereby reducing unnecessary computations.

Procedure / Algorithm with Complexities :
Algorithm

1. Start the program.
2. Read the number of cities **n**.
3. Input the **cost matrix** representing distances between cities.
4. Start from the first city and mark it as visited.
5. Generate possible paths by visiting unvisited cities.
6. Calculate the cost of each path.
7. If the cost is less than the current minimum, continue exploring.
8. Otherwise discard the path.
9. Repeat until all paths are evaluated.
10. Display the minimum cost tour.
11. Stop the program.

\
Time Complexity
Worst Case : O(n!)
Space Complexity $O(n^2)$

Source Code :
```
#include <stdio.h>
#include <limits.h>
#define MAX 10

int cost[MAX][MAX];
int visited[MAX];
int path[MAX], bestPath[MAX];

int n,i,j;
int minCost = INT_MAX;

void tsp(int city, int count, int currentCost);
```

```
int main()
{
   printf("Enter number of cities: ");
   scanf("%d",&n);

   printf("Enter cost matrix:\n");

   for(i=0;i<n;i++)
      for(j=0;j<n;j++)
         scanf("%d",&cost[i][j]);

   visited[0] = 1;
   path[0] = 0;

   tsp(0,1,0);

   printf("\nMinimum Travelling Cost = %d\n",minCost);

   printf("Optimal Tour: ");

   for(i=0;i<n;i++)
      printf("%d -> ",bestPath[i]);

   printf("0\n");

   return 0;
}

void tsp(int city, int count, int currentCost)
{
   if(count == n)
   {
      int total = currentCost + cost[city][0];

      if(total < minCost)
      {
         minCost = total;

         for(i=0;i<n;i++)
            bestPath[i] = path[i];
      }
```

```
        return;
    }

    for(i=0;i<n;i++)
    {
        if(!visited[i] && cost[city][i] != 0)
        {
            int tempCost = currentCost + cost[city][i];

            if(tempCost < minCost)
            {
                visited[i] = 1;
                path[count] = i;

                tsp(i, count+1, tempCost);

                visited[i] = 0;
            }
        }
    }
}
```

Input & Output :

```
Enter number of cities: 4

Enter cost matrix:
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Output
Minimum Travelling Cost = 80
Optimal Tour: 0 -> 1 -> 3 -> 2 -> 0
```

Result :

Thus, the program to solve the **Travelling Salesperson Problem using the Branch and Bound approach** was successfully implemented and executed.